

Lucas Zacarias Martins Neves

**Deteccção de vulnerabilidades de memória em C:
Abordagem integrando VSA e HOFG**

São Paulo

2025

Lucas Zacarias Martins Neves

Detecção de vulnerabilidades de memória em C: Abordagem integrando VSA e HOFG

Proposta de Trabalho de Conclusão de Curso
apresentado ao SENAC Santo Amaro como
requisito parcial para aprovação na disciplina
de TCC1 do curso de Ciência da computação.

SENAC – Serviço Nacional de Aprendizagem Comercial
Unidade Santo Amaro
Bacharelado em Ciência da computação

Orientador: Prof. Nome do Orientador

São Paulo
2025

Resumo

Palavras-chave: Vazamento de memória, HOFG, VSA.

Sumário

1	INTRODUÇÃO	4
1.1	Contexto	4
1.2	Justificativa	4
1.3	Hipótese	5
1.4	Objetivos	5
1.4.1	Objetivo geral	5
1.4.2	Objetivos específicos	6
2	REVISÃO BIBLIOGRÁFICA	7
2.1	Referencial teórico	7
2.1.1	Memória <i>Heap</i>	7
2.1.2	Análise estática	7
2.1.2.1	Árvore sintática abstrata (AST)	7
2.1.2.2	Forma de atribuição única estática (SSA)	8
2.1.2.3	<i>Value-Set Analysis</i>	8
2.1.2.4	Limitações da análise estática	9
2.1.3	LLVM	9
2.1.3.1	Implementação do Design de três fases	10
2.1.3.2	Representação Intermediária (LLVM IR)	10
2.1.4	Vulnerabilidades de memória	11
2.1.4.1	Common Weakness Enumeration (CWE)	11
2.1.4.2	CWE-401	12
2.1.4.3	CWE-415	12
2.1.4.4	CWE-416	12
2.2	Metodologia da pesquisa bibliográfica	13
2.3	Métricas de Avaliação	13
2.4	Fundamentos	13
2.5	Trabalhos relacionados	13
3	DESENVOLVIMENTO	14
3.1	Solução proposta	14
3.2	Cronograma de trabalho	14
	REFERÊNCIAS	15

1 Introdução

1.1 Contexto

Em um levantamento realizado pela *Microsoft Security Response Center* (MSRC) realizado em 2019 (THOMAS, 2019), foi acurado que aproximadamente 70% das vulnerabilidades corrigidas em softwares da multinacional entre o período de 2004 e 2018 tem ligação a falhas de segurança de memória nas linguagens C e C++, causadas por desenvolvedores que inseriram tais erros em seus códigos sem perceber. Além disso, a situação teve piora não somente com o incremento da base de código da empresa, mas também com a adição de mais código aberto aos seus projetos.

É pontuado ainda pela Microsoft que existem métodos para auxiliar os desenvolvedores a desenvolver código seguro, tais métodos variam desde ferramentas e algoritmos desenvolvidos para análise do próprio código escrito até a guias e metodologias de desenvolvimento para software seguro, treinamentos internos, reuniões para monitoria e análise de código. Salvaguardas que atuam em todas as camadas do desenvolvimento de software, e mesmo com tais itens, tais vulnerabilidades de memória continuam predominantes.

Dando destaque às ferramentas de análise de código estático, por vezes exigem tempo para que os desenvolvedores aprendam a fazer uso do analisador, cenário este que torna o uso de tais ferramentas algo inviável quando o tempo não é aliado. Além disso, tais ferramentas não garantem que todas as vulnerabilidades e possíveis falhas de um código sejam encontradas, pois focam numa análise do código fonte do software, sendo escrito numa linguagem de alto nível e não na linguagem de máquina gerada pelo compilador que será executada pelo hardware (BALAKRISHNAN; REPS, 2010).

Além de métodos de código estático, outra opção também pontuada pela MSRC para a diminuição de ocorrências, seria o uso de linguagens de programação *memory-safe*, ou seja, que retiram das mãos de desenvolvedores os ônus de preocupações com segurança de memória e a adicionam na própria linguagem, fazendo gerenciamento de memória ou empregando regras estritas na fase de compilação, evitando tais vulnerabilidades. Um exemplo de linguagem de programação *memory-safe* é o Rust, sendo desenvolvido pela Mozilla e que tem como um de seus princípios a segurança de memória.

1.2 Justificativa

Mesmo com a existência de linguagens de programação como Rust, sendo focado em segurança, existem projetos que apresentariam custos excessivos para portabilidade entre as linguagens, fazendo com que seja necessário o uso da linguagem original do

desenvolvimento, como o C, neste caso. Com isso, para garantir maior segurança de tais projetos, um analisador de código pode ser desenvolvido como uma das soluções, com foco em encontrar potenciais vazamentos e problemas provenientes da má gestão de itens armazenados na memória.

1.3 Hipótese

A hipótese do projeto esta numa integração entre dois métodos de análise estática.

O primeiro método é o *Heap Object Flow Graph* (HOFG) (C; CHERAMANGALATH, 2023), sendo uma representação do fluxo dos blocos de memória alocados na Heap, na qual os vértices do grafo são obtidos a partir das variáveis do programa que são ponteiros para a memória Heap. A partir do HOFG foi desenvolvido o *HOFG-Analyser*, uma ferramenta que utiliza o código fonte convertido para o formato HOFG para verificar vulnerabilidades de memória.

Porém, o *HOFG-Analyser* obteve 14% de falsos positivos em seus testes, apresentando dificuldades com aritmética de ponteiros presente na linguagem C. Para lidar com as dificuldades presentes pela aritmética de ponteiros e ser possível diminuir a taxa de falsos positivos do HOFG original, uma integração com o *Value-Set Analysis* (VSA) (BALAKRISHNAN, 2007) poderia ser realizada. O VSA, técnica de interpretação abstrata originalmente aplicada para análise de executáveis, tem o objetivo de encontrar uma aproximação segura para o conjunto de valores que cada objeto de dados pode armazenar em cada ponto de um programa.

Ao integrar VSA ao HOFG, seria possível acompanhar o endereço abstrato dos objetos alocados na memória Heap, bem como estimar limites de intervalos que tais endereços podem assumir, permitindo assim acompanhar as mudanças de estado dos objetos com mais profundidade, encontrando atribuições que possam, por sua vez, gerar vulnerabilidades de memória.

1.4 Objetivos

1.4.1 Objetivo geral

O objetivo principal do trabalho é implementar um analisador estático de código para a linguagem C, com foco na detecção de vulnerabilidades de memória, através da integração da representação *Heap Object Flow Graph* (HOFG) com técnicas de *Value-Set Analysis* (VSA) para o rastreamento preciso de ponteiros (*offset* e *range*).

1.4.2 Objetivos específicos

1. Fundamentar teoricamente a integração do *Value-Set analysis* à estrutura do *Heap Object Flow Graph* (HOFG), demonstrando como essa abordagem conjunta pode mitigar as limitações de rastreamento de ponteiros e reduzir a taxa de falsos positivos da análise estática.
2. Implementar motor de análise estática processando a representação intermediária LLVM IR, construindo os algoritmos necessários para a extração e geração do HOFG e sua integração com o VSA.
3. Avaliar a precisão e a eficiência do analisador implementado utilizando conjuntos de testes que exploram falhas da linguagem C (como vazamentos de memória e usos após desalocação).
4. Comparar os resultados obtidos com referências comparativas tendo como foco principal a taxa de falsos positivos e verdadeiro positivos, e comparar o tempo e custo computacional da implementação original do HOFG, para validar o impacto da integração.

2 Revisão Bibliográfica

2.1 Referencial teórico

2.1.1 Memória *Heap*

A memória *Heap* é uma área da memória virtual de um processo utilizada para a alocação dinâmica de memória. Tal região tem início logo após a área de dados não inicializados do programa e cresce para cima, em direção a endereços de memória mais altos. Para cada processo, o *kernel* mantém uma variável *brk* (sendo pronunciada *break*, ou quebra) que aponta para o topo da *Heap* (BRYANT; O'HALLARON, 2010). Um alocador dinâmico gerencia a *Heap* como uma coleção de blocos de tamanhos variados. Cada bloco é um segmento (*chunk*) contínuo de memória virtual que pode estar em dois estados: alocado ou livre. Um bloco alocado é reservado para uso de forma explícita pela aplicação, permanecendo neste estado até que seja liberado, tanto de forma explícita, pelo aplicativo, quanto de forma implícita, pelo próprio alocador de memória (como em sistemas com *Garbage Collection*). E, por sua vez, um bloco livre está disponível para ser alocado, e permanecesse assim até que, de forma explícita, seja alocado através de uma nova requisição de alocação feita pela aplicação. O fato de que, por vezes, o tamanho exato de certas estruturas de dados que serão utilizados em um programa só se tornam conhecidos até que o mesmo seja executado se torna o principal motivo para a utilização da alocação dinâmica de memória e, conseqüentemente, da memória *Heap* (BRYANT; O'HALLARON, 2010).

2.1.2 Análise estática

A análise estática é uma técnica para obter informações sobre o comportamento em tempo de execução de um programa, porém sem a necessidade de executar o programa. Isso ocorre pois os métodos de análise estática exploram o comportamento do programa a partir de abstrações e aproximações para mapear o espaço de estados do programa, dessa forma detectando comportamentos tais como vulnerabilidades e fluxos inválidos para qualquer entrada possível.

2.1.2.1 Árvore sintática abstrata (AST)

Segundo Aho et al. (2009), as árvores sintáticas abstratas (ASTs), ou simplesmente árvores sintáticas, possuem para uma expressão, cada nó interno de uma expressão representa um operador, enquanto os seus nós filhos representam os respectivos operandos. Além disso, as ASTs se assemelham às árvores de derivação até certo nível, diferenciando-se

pelo fato de que os nós interiores da AST representam construções de programação, ao passo que na árvore de derivação os nós interiores representam símbolos não-terminais. Outra diferença é a ausência de símbolos não-terminais auxiliares, tais como aqueles que representam termos, fatores, ou outras variações de expressões, na AST. Essa falta ocorre pois tais detalhes sintáticos são desnecessários para a fase de tradução, resultando em uma representação mais condensada e focada na semântica do código fonte.

2.1.2.2 Forma de atribuição única estática (SSA)

O SSA (*Static Single Assignment*) é uma forma de representação de código intermediário, tem como ponto chave as características de que cada definição de variável possui um nome distinto, e de que cada uso refere-se a apenas uma única definição, sendo essas também as duas restrições da representação, significando que, caso um programa atenda tais restrições, tal programa estará em sua forma SSA. Para garantir essa singularidade, variáveis que sofrem múltiplas atribuições no código-fonte são renomeadas, sendo criadas versões distintas para cada nova atribuição (por exemplo, v1, v2, v3) (SINGER, 2022).

Além disso, em programas com desvios do fluxo de controle, contendo blocos condicionais e laços de repetição, diferentes versões da mesma variável podem chegar ao mesmo ponto de convergência no Grafo de Fluxo de Controle (CFG). São inseridas então, nesses pontos de junção, as chamadas funções (*phi functions*), possuindo N argumentos correspondentes aos N caminhos do fluxo de controle convergentes, e seleciona a versão correta da variável com base no caminho de execução tomado pelo programa. Sendo criada inicialmente para facilitar o desenvolvimento de transformações de programa de alto nível, a forma ganhou popularidade pelas suas propriedades que permitem a simplificação de algoritmos e a redução da complexidade computacional das análises de fluxo de dados. Atualmente, é amplamente utilizada por compiladores de código aberto e comerciais, como é o caso do LLVM, sendo uma estrutura fundamental para a análise estática de programas (SINGER, 2022).

2.1.2.3 Value-Set Analysis

O *Value-Set Analysis* (VSA) é uma técnica de interpretação abstrata de código executável, que tem como objetivo encontrar uma aproximação segura para o conjunto de valores que cada objeto de dados pode armazenar em cada ponto de um programa (BALAKRISHNAN, 2007). O VSA possui similaridades com o problema de análise de ponteiros (*pointer-analysis*), frequentemente estudado em análises de programas escritos em linguagens de alto nível. Enquanto a análise de ponteiros tradicional determina uma sobreaproximação (*over-approximation*) de conjunto de variáveis cujos endereços uma determinada variável pode conter, o VSA atua determinando uma sobreaproximação do conjunto de endereços que cada objeto de dados pode conter em cada ponto do programa.

Como consequência, os resultados do VSA podem ser utilizados para identificar quais *a-locs* (*abstract locations*) estão contidos nos endereços de um determinado *a-loc*. Por outro lado, o VSA também possui características de análises estáticas numéricas. Além de rastrear endereços, o algoritmo determina uma sobreaproximação do conjunto de valores inteiros que cada objeto de dados pode assumir ao longo da execução (BALAKRISHNAN, 2007).

2.1.2.4 Limitações da análise estática

Os métodos de análise estática não garantem que todas as vulnerabilidades de um código sejam encontradas. Dentre os motivos, sendo listados por Balakrishnan (2007), estão:

- O uso de bibliotecas já compiladas (tais como as DLLs) em programas, por vezes sem estar disponíveis em sua forma fonte. Gerando a necessidade de realizar esboços que simulam os efeitos das chamadas de função de tais bibliotecas. Por não poder realizar a análise em tais bibliotecas, é possível que a mesma possua erros, não sendo detectados durante a análise estática sendo realizada, resultando em inconsistências.
- O fato de que códigos fonte são modificados pelo compilador durante a compilação, realizando otimizações e adicionando instrumentações de código, sendo itens não encontrados pelas ferramentas que analisam tais códigos.
- O código fonte de um programa pode ser escrito em mais de uma linguagem de programação, tornando mais dificultosa o design de tais ferramentas de análise, pois precisam dar suporte a múltiplas linguagens de programação, cada uma contendo suas características e peculiaridades.
- E, mesmo em um código fonte escrito em apenas uma linguagem de programação de alto nível, tal código pode conter código escrito em *assembly* em certos locais. Ferramentas de análise estática normalmente ignoram o código *assembly* adicionado, ou não levam a análise além do código *assembly* adicionado.

2.1.3 LLVM

Segundo Lattner e Adve (2004), o LLVM é uma infraestrutura de compilação designada para prover transparência e análise e transformação de programas de forma persistente durante todo o seu ciclo de vida. O sistema é fundamentado em uma representação intermediária (IR) de baixo nível, baseada na sua forma SSA, permitindo uma ponte universal entre diferentes linguagens e arquiteturas. A arquitetura introduz recursos fundamentais para a análise de código:

- Um simples sistema de tipagem independente de linguagem que expõe os tipos primitivos usados comumente para implementar atributos de linguagens de alto nível.
- Instrução para endereçamento aritmético, sendo diferente do C puro, o LLVM utiliza uma instrução específica para cálculos de ponteiros que preserva informações de tipo e limites de estruturas, sendo essencial para detectar acessos inválidos.
- Mecanismo uniforme de exceções, utilizado para implementar recursos de tratamento de exceções comumente encontrados em linguagens de alto nível de forma eficiente.

2.1.3.1 Implementação do Design de três fases

Num compilador moderno, três componentes principais são aplicados ([LATTNER, 2011](#)), sendo eles:

- O front end, responsável por analisar o código fonte, chegar erros e construir a árvore de sintaxe abstrata (AST) específica para a linguagem em questão.
- O otimizador, responsável por realizar transformações de código numa tentativa de melhorar o tempo de execução do código, realizando, por exemplo, remoções de itens redundantes que seriam computados.
- O back end, também conhecido como o gerador de código. Responsável por mapear o código para o conjunto de instruções da arquitetura alvo. Dentre os itens comumente encontrados no back end, estão inclusos a seleção de instruções, alocação de registradores e o escalonamento de instruções.

Com tal padrão a portabilidade de um compilador para uma nova linguagem de programação fonte gera a necessidade de implementação do novo *front end*, mas permitindo que o otimizador e o *back end* sejam reutilizados. Porém, esse benefício não fora alcançado em todos os casos já que implementações do padrão de três fases, numa época em que o LLVM estava tendo seu início, em linguagens como Perl, Python, Ruby e Java não compartilhavam código entre si, ou seja, cada uma destas linguagens possuía sua própria implementação de *front end*, otimizador e *back end* ([LATTNER, 2011](#)).

2.1.3.2 Representação Intermediária (LLVM IR)

A representação intermediária é o aspecto principal do LLVM, sendo a forma que compilador utiliza para representar o código fonte sendo compilado. É designada para análises de nível intermediário e transformações que são encontradas no setor de otimização de um compilador. Desenhada com objetivos de adicionar suporte para otimizações em tempo de execução não custosas, otimizações interprocedurais, análise completa do

programa, transformações abrangendo nível de reestruturação, dentre outros, tem como aspecto mais importante o fato de ser definida como uma linguagem de primeira classe com semântica bem definida, tendo seu conjunto de instruções próximo do padrão RISC e possuindo como um de seus pilares a forma SSA ([LATTNER, 2011](#)).

Sendo a única interface para o otimizador do LLVM, se torna o único requisito para escrever um front end para a infraestrutura. Por possuir uma forma textual legível, desenvolver um front end que tem como saída a representação intermediária do LLVM se torna algo possível e viável. Tal característica é um dos principais fatores para a disseminação da infraestrutura do LLVM em diferentes aplicações, já que a completude presente na representação permite que o suporte de uma nova linguagem seja simplificado ([LATTNER, 2011](#)).

2.1.4 Vulnerabilidades de memória

Segundo o *Microsoft Security Response Center* (MSRC) ([THOMAS, 2019](#)), entre o período de 2004 a 2018 aproximadamente 70% das vulnerabilidades corrigidas mapeadas a um CVE foram causadas por desenvolvedores que de forma desavisada adicionaram problemas relacionados a vulnerabilidades de memória em seus códigos escritos nas linguagens C e C++. Além disso, a medida que a base de código da Microsoft era incrementada e mais código aberto era utilizado, tal problema continuava a aumentar.

2.1.4.1 Common Weakness Enumeration (CWE)

A *Common Weakness Enumeration* (CWE) consiste em uma lista desenvolvida de forma colaborativa, que relaciona fraquezas recorrentes em software e hardware. No contexto de segurança, uma fraqueza (weakness) é uma condição num software, firmware, hardware, ou serviço que, a partir de certas circunstâncias, pode contribuir para a introdução de vulnerabilidades. Fraquezas são identificadas, classificadas, descritas e integradas à lista oficial da CWE (*CWE List*). A compreensão dessas fraquezas que resultam em vulnerabilidades permite que desenvolvedores de software, designers de hardware, e arquitetos de segurança atuem na mitigação e eliminação de tais fraquezas ainda nas fases iniciais do ciclo de desenvolvimento ([The MITRE Corporation, 2026](#)).

A lista CWE é atualizada de três a quatro vezes por ano, incluindo novas descobertas e atualizando itens registrados anteriormente. Antes da publicação oficial, o conteúdo em desenvolvimento é disponibilizado e pode ser acompanhado através do *CWE Content Development Repository* (CDR), sendo um repositório público com objetivo de garantir transparência e colaboração da comunidade, viabilizando a submissão de novas fraquezas, contribuições para registros existentes e acompanhamento das operações internas da CWE ([The MITRE Corporation, 2026](#)).

2.1.4.2 CWE-401

Conforme a [OWASP Foundation \(2026\)](#), um vazamento de memória é uma forma não intencionada de consumo de memória, ocorrendo quando o desenvolvedor falha em liberar um bloco de memória alocada quando a mesma não é mais necessária. Sendo dividido, de forma geral, em três cenários:

- Aplicações de curta duração (*Short Lived User-Land Application*), tendo pouco ou nenhum impacto notável, levando em consideração que sistemas operacionais modernos obtêm a memória perdida após o programa ser encerrado.
- Aplicações de longa duração (*Long Lived User-Land Application*), tendo riscos potenciais já que tais programas continuam perdendo memória ao longo do tempo, causando, eventualmente, o consumo integral dos recursos da memória RAM.
- Processos em nível de núcleo (*Kernel-land Process*), sendo o caso com nível mais alto de consequências, trazendo vazamentos de memória a nível de *Kernel*, gerando problemas de estabilidade voltados ao sistema.

2.1.4.3 CWE-415

Com o título de liberação dupla (*double free*), ocorre quando o mesmo endereço de memória é liberado mais de uma vez. Isso acontece pois, quando o programa chama a função de liberação (como a função `free()` da biblioteca `stdlib`, na linguagem C) utilizando o mesmo endereço de memória como argumento, a estrutura de dados responsável por gerenciar a heap pode ser corrompida, permitindo assim que um usuário malicioso escreva em posições arbitrárias de memória. O impacto de tal corrupção pode causar desde a falha abrupta do programa até a alteração do fluxo de execução. Ao sobrescrever registradores ou endereços específicos de memória, um atacante pode alterar o programa, sendo capaz de injetar e executar códigos arbitrários, resultando, por exemplo, na escalada de privilégios sobre o sistema em um emulador de terminal (OWASP, 2026).

2.1.4.4 CWE-416

O *Use After Free* é a referência de um endereço de memória que foi desalocado, tal ato gera consequências imprevisíveis no sistema, com resultados que variam desde uma falha abrupta do programa até a execução de códigos e comandos arbitrários, dependendo do contexto e do momento que a vulnerabilidade ocorre. A reutilização de um endereço desalocado, segundo o *Open Web Application Security Project* (OWASP, 2026), é a forma mais simples de corrupção de dados, visto que, neste cenário, o bloco de memória anteriormente liberado pode ser realocado para um novo objeto válido ao longo da execução do programa. Com isso o ponteiro original ainda aponta para esse endereço (se tornando um

ponteiro pendente, ou dangling pointer), qualquer alteração feita através dele corromperá os dados da nova alocação legítima, gerando a falha imprevisível.

2.2 Metodologia da pesquisa bibliográfica

Design Science Research

2.3 Métricas de Avaliação

2.4 Fundamentos

2.5 Trabalhos relacionados

O analisador será avaliado a partir do Juliet Test Suite, sendo um conjunto de casos da linguagem C que exploram as suas vulnerabilidades. O resultado obtido será comparado com os resultados de analisadores e algoritmos desenvolvidos que possuem como finalidade o mesmo objetivo, ou seja, de garantir a segurança dos projetos. O principal analisador a ser comparado é o *HOFG-Analyser*, sendo a aplicação original da representação de programas.

3 Desenvolvimento

3.1 Solução proposta

3.2 Cronograma de trabalho

Referências

- AHO, A. V. et al. *Compiladores: Princípios, Técnicas e Ferramentas*. 2. ed. [S.l.]: Pearson Addison-Wesley, 2009. Tradução de Daniel Vieira. 7
- BALAKRISHNAN, G. PhD thesis, *WYSINWYX: What You See Is Not What You Execute*. Madison, WI: [s.n.], 2007. Disponível em: <https://research.cs.wisc.edu/wpis/papers/balakrishnan_thesis.pdf>. 5, 8, 9
- BALAKRISHNAN, G.; REPS, T. Wysinwyx: What you see is not what you execute. *ACM Trans. Program. Lang. Syst.*, Association for Computing Machinery, New York, NY, USA, v. 32, n. 6, ago. 2010. ISSN 0164-0925. Disponível em: <<https://doi.org/10.1145/1749608.1749612>>. 4
- BRYANT, R. E.; O'HALLARON, D. R. *Computer Systems: A Programmer's Perspective*. 2. ed. Upper Saddle River, NJ: Prentice Hall, 2010. 7
- C, H. M.; CHERAMANGALATH, U. Memory leak detection using heap object flow graph (hofg). Association for Computing Machinery, New York, NY, USA, 2023. Disponível em: <<https://doi.org/10.1145/3578527.3578528>>. 5
- LATTNER, C. LLVM. In: BROWN, A.; WILSON, G. (Ed.). *The Architecture of Open Source Applications*. Lulu.com, 2011. v. 1, cap. 11. Acesso em: 13 abr. 2026. Disponível em: <<https://aosabook.org/en/v1/llvm.html>>. 10, 11
- LATTNER, C.; ADVE, V. Llm: A compilation framework for lifelong program analysis & transformation. In: *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization*. USA: IEEE Computer Society, 2004. (CGO '04), p. 75. ISBN 0769521029. 9
- OWASP Foundation. *Memory leak*. 2026. Acesso em: 13 abr. 2026. Disponível em: <https://owasp.org/www-community/vulnerabilities/Memory_leak>. 12
- SINGER, J. Introduction. In: RASTELLO, F.; Bouchez Tichadou, F. (Ed.). *SSA-based Compiler Design*. [S.l.]: Springer International Publishing, 2022. ISBN 978-3-030-80515-9. 8
- The MITRE Corporation. *About CWE*. 2026. Acesso em: 13 abr. 2026. Disponível em: <<https://cwe.mitre.org/about/index.html>>. 11
- THOMAS, G. *A proactive approach to more secure code*. 2019. Acesso em: 13 abr. 2026. Disponível em: <<https://www.microsoft.com/en-us/msrc/blog/2019/07/a-proactive-approach-to-more-secure-code>>. 4, 11